

Minisql实验报告

§ Chapter 1 - 系统概述

I 任务概叙

实验目的与需求

1. 目的

设计并实现一个精简型单用户SQL引擎MiniSQL，允许用户通过字符界面输入SQL语句实现基本的增删改查操作，并能够通过索引来优化性能。

2. 数据类型

要求支持三种基本数据类型 **integer**, **char(n)**, **float**。

3. 表定义

一个表可以定义**32个属性**，各属性可以指定是否为 **unique**，支持单属性的**主键**定义。

4. 索引定义

对于表的**主属性自动建立B+树索引**，对于声明为**unique的属性**也需要建立**B+树索引**。

5. 数据操作

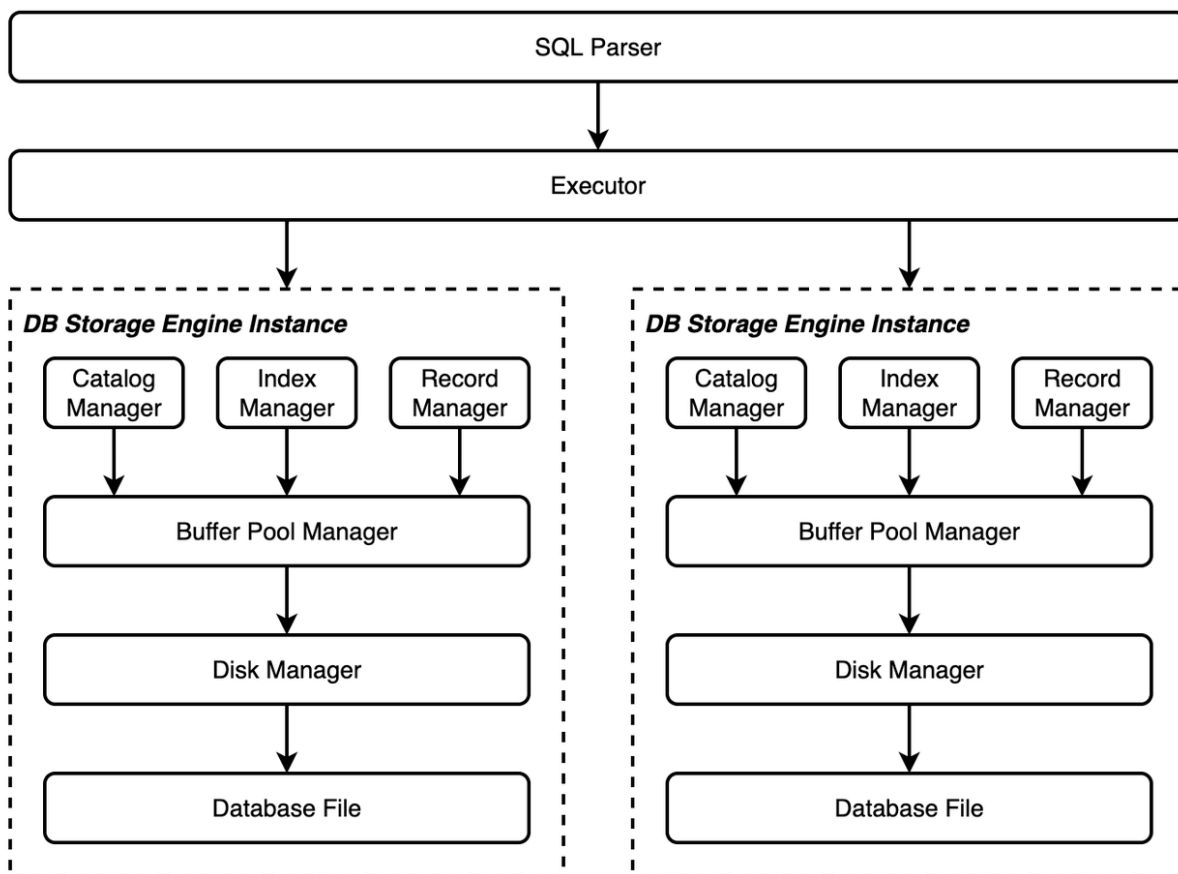
可以通过 **and** 或 **or** 连接的**多个条件**进行查询，支持**等值查询**和**区间查询**。支持**每次一条记录**的插入操作；支持**每次一条或多条记录**的删除操作。

6. 在工程实现上，使用源代码管理工具（如Git）进行代码管理，代码提交历史和每次提交的信息清晰明确，我们使用 ZJUGit 仓库进行管理，能够较为高效的合并代码；同时编写的代码应符合代码规范，具有良好的代码风格。

II 系统架构

在系统架构中，**解释器SQL Parser**在解析SQL语句后将生成的语法树交由**执行器Executor**处理。执行器则根据语法树的内容对相应的数据库实例（DB Storage Engine Instance）进行操作。

每个DB Storage Engine Instance对应了一个数据库实例（即通过CREATE DATABASE创建的数据库）。在每个数据库实例中，用户可以定义若干**表**（**tableInfo**）和**索引**（**indexInfo**），表和索引的信息通过**Catalog Manager**、**Index Manager**和**Record Manager**进行维护。使用不同的数据库实例可以通过USE语句切换。



§ Chapter 2 – 框架详细设计

1 DISK AND BUFFER POOL MANAGER

1.1 模块概述

在 MiniSQL 的设计中，Disk Manager 和 Buffer Pool Manager 模块位于架构的最底层。Disk Manager 主要负责数据库文件中数据页的分配和回收，以及数据页中数据的读取和写入。其中，数据页的分配和回收通过位图（Bitmap）这一数据结构实现，位图中每个比特（Bit）对应一个数据页的分配情况，用于标记该数据页是否空闲（0 表示空闲，1 表示已分配）。

当 Buffer Pool Manager 需要向 Disk Manager 请求某个数据页时，Disk Manager 会通过某种映射关系，找到该数据页在磁盘文件中的物理位置，将其读取到内存中返还给 Buffer Pool Manager。而 Buffer Pool Manager 主要负责将磁盘中的数据页从内存中来回移动到磁盘，这使得我们设计的数据库管理系统能够支持那些占用空间超过设备允许最大内存空间的数据库。

1.2 功能实现

1. 位图页的实现

功能介绍

位图页是Disk Manager模块中的一部分，是实现磁盘页分配与回收工作的必要功能组件。位图页与数据页一样，占用 `PAGE_SIZE`（4KB）的空间，标记一段连续页的分配情况。

Bitmap Page由两部分组成，一部分是用于加速Bitmap内部查找的元信息（Bitmap Page Meta），其包含当前已经分配的页的数量（`page_allocated_`）以及下一个空闲的数据页(`next_free_page_`)，除去元信息外，页中剩余的部分就是Bitmap存储的具体数据。

相关函数

- `BitmapPage::AllocatePage(&page_offset)`：分配一个空闲页，并通过 `page_offset` 返回所分配的空闲页位于该段中的下标（从 0 开始）；
- 位图页内部实现均采用位操作,比如要将第1234个位置置为1,先计算字节序: $1234 \gg 3 = 154$, 位序: $0x80 \gg (1234 \& 0x07) = 2$, 那么 1234 放在 M 的下标 154 字节处, 把该字节的 2 号位置为 1
- `BitmapPage::DeAllocatePage(page_offset)`：回收已经被分配的页；
 - `BitmapPage::IsPageFree(page_offset)`：判断给定的页是否是空闲（未分配）的。

2. 磁盘数据页管理

功能介绍

在实现位图页后，我们可以通过一个位图页加上一段连续的数据页（数据页的数量取决于位图页最大能够支持的比特数）来对磁盘文件（DB File）中数据页进行分配和回收。而我们把一个位图页加一段连续的数据页看成数据库文件中的一个分区，再通过一个额外的元信息页来记录这些分区的信息，可以使磁盘文件能够维护更多的数据页信息，而不仅仅是32768个数据页，结构如下：

Disk Meta Page	Extent Meta (Bitmap Page)	Extent Pages	Extent Meta (Bitmap Page)	Extent Pages	...
----------------	---------------------------	--------------	---------------------------	--------------	-----

由于元数据所占用的数据页实际上是不存储数据库数据的，而它们实际上又占据了数据库文件中的数据页，换言之，实际上真正存储数据的数据页是不连续的，如下表：

物理页号	0	1	2	3	4	5	6	...
职责	磁盘元数据	位图页	数据页	数据页	数据页	位图页	数据页	
逻辑页号	/	/	0	1	2		3	

为了使得上层的Buffer Pool Manager对于Disk Manager中的页分配保持无感知，我们还需实现对页号做映射（映射成上表中的逻辑页号）的函数。

相关函数

- `DiskManager::AllocatePage()`：从磁盘中分配一个空闲页，并返回空闲页的**逻辑页号**；
- `DiskManager::DeAllocatePage(logical_page_id)`：释放磁盘中**逻辑页号**对应的物理页。
- `DiskManager::IsPageFree(logical_page_id)`：判断该**逻辑页号**对应的数据页是否空闲。

- `DiskManager::MapPageId(logical_page_id)`：可根据需要实现。在 `DiskManager` 类的私有成员中，该函数可以用于将逻辑页号转换成物理页号。

3. LRU 替换策略

功能介绍

Buffer Pool Replacer负责跟踪Buffer Pool中数据页的使用情况，并在Buffer Pool没有空闲页时决定替换哪一个数据页。本部分要求实现一个基于LRU替换算法的 `LRUReplacer`，同时将实现一种新的缓冲区替换算法（如Clock Replacer）作为bonus（已实现）。

相关函数

- `LRUReplacer::Victim(*frame_id)`：替换（即删除）与所有被跟踪的页相比最近最少被访问的页，将其页帧号（即数据页在Buffer Pool的Page数组中的下标）存储在输出参数 `frame_id` 中输出并返回 `true`，如果当前没有可以替换的元素则返回 `false`；
- `LRUReplacer::Pin(frame_id)`：将数据页固定使之不能被 `Replacer` 替换，即从 `lru_list_` 中移除该数据页对应的页帧。`Pin` 函数应当在一个数据页被Buffer Pool Manager固定时被调用；
- `LRUReplacer::Unpin(frame_id)`：将数据页解除固定，放入 `lru_list_` 中，使之可以在必要时被 `Replacer` 替换掉。`Unpin` 函数应当在一个数据页的引用计数变为 0 时被Buffer Pool Manager调用，使页帧对应的数据页能够在必要时被替换；
- `LRUReplacer::Size()`：此方法返回当前 `LRUReplacer` 中能够被替换的数据页的数量。

4. 缓冲池管理

功能介绍

Buffer Pool Manager负责从Disk Manager中获取数据页并将它们存储在内存中，并在必要时将脏页面转储到磁盘中（如需要为新的页面腾出空间）。

数据库系统中，所有内存页面都由Page对象记录有关信息。同一个Page对象在系统的整个生命周期内，可能会对应很多不同的物理页。Page对象的唯一标识符`page_id`用于跟踪它所包含的物理页，如果Page对象不包含物理页，那么`page_id`必须被设置为`INVALID_PAGE_ID`。每个Page对象还维护了一个计数器`pin_count_`，它用于记录固定(Pin)该页面的线程数。Buffer Pool Manager将不允许释放已经被固定的Page。每个Page对象还将记录它是否脏页，在复用Page对象之前必须将脏的内容转储到磁盘中。

相关函数

- `BufferPoolManager::FetchPage(page_id)`：根据逻辑页号获取对应的数据页，如果该数据页不在内存中，则需要从磁盘中进行读取；

1. Search the page table for the requested page (P).
 1. If P exists, pin it and return it immediately.
 2. If P does not exist, find a replacement page (R) from either the free list or the replacer. Note that pages are always found from the free list first.
2. If R is dirty, write it back to the disk.
3. Delete R from the page table and insert P.
4. Update P's metadata, read in the page content from disk, and then return a pointer to P.

- `BufferPoolManager::NewPage(&page_id)`：分配一个新的数据页，并将逻辑页号于 `page_id` 中返回；
 0. Remember to call `AllocatePage`.
 1. If all the pages in the buffer pool are pinned, return `nullptr`.
 2. Pick a victim page `P` from either the free list or the replacer. Always pick from the free list first.
 3. Update `P`'s metadata, zero out memory and add `P` to the page table.
 4. Set the page ID output parameter. Return a pointer to `P`.
- `BufferPoolManager::UnpinPage(page_id, is_dirty)`：取消固定一个数据页；
- `BufferPoolManager::FlushPage(page_id)`：将数据页转储到磁盘中；
- `BufferPoolManager::DeletePage(page_id)`：释放一个数据页；
 0. Make sure you call `DeallocatePage`!
 1. Search the page table for the requested page (`P`).
 1. If `P` does not exist, return `true`.
 2. If `P` exists, but has a non-zero pin-count, return `false`. Someone is using the page.
 3. Otherwise, `P` can be deleted. Remove `P` from the page table, reset its metadata and return it to the free list.
- `BufferPoolManager::FlushAllPages()`：将所有的页面都转储到磁盘中。

1.3 Bonus

该模块需要实现一种新的缓冲区替换算法（如Clock Replacer）作为bonus，在此实现了Clock Replacer，并为它写段两个测试代码，第一段是对标框架中的lru Replacer的测试，第二个是百万级别的压力测试，均成功通过。

具体代码分析见个人报告

框架建议部分见个人报告

2 RECORD MANAGER

2.1 模块概述

在 MiniSQL 的设计中，Record Manager负责管理数据表中所有的记录，它能够支持记录的插入、删除与查找操作，并对外提供相应的接口。

与记录（Record）相关的概念有以下几个：

- 列（Column）：在 `src/include/record/column.h` 中被定义，用于定义和表示数据表中的某一个字段，即包含了这个字段的字段名、字段类型、是否唯一等等；
- 模式（Schema）：在 `src/include/record/schema.h` 中被定义，用于表示一个数据表或是一个索引的结构。一个 Schema 由一个或多个的 Column 构成；

- 域 (Field) : 在 `src/include/record/field.h` 中被定义, 它对应于一条记录中某一个字段的数据信息, 如存储数据的数据类型, 是否是空, 存储数据的值等等;
- 行 (Row) : 在 `src/include/record/row.h` 中被定义, 与元组的概念等价, 用于存储记录或索引键, 一个 Row 由一个或多个 Field 构成。

2.2 功能实现

1. 实现数据的序列化与反序列化

功能介绍

为了能够持久化存储上面提到的 Row、Field、Schema 和 Column 对象, 我们需要提供一种能够将这些对象序列化成字节流 (char*) 的方法, 以写入数据页中。与之相对, 为了能够从磁盘中恢复这些对象, 我们同样需要能够提供一种反序列化的方法, 从数据页的 char* 类型的字节流中反序列化出我们需要的对象。总而言之, 序列化和反序列化操作实际上是将数据库系统中的对象 (包括记录、索引、目录等) 进行内外存格式转化的过程。

在本章中 Field 类型对象的序列化和反序列化操作已经提供。

相关函数

1. row 的正反序列化:

Row 是由 field 组成, 序列化 row 即将 row 内的全部 field 进行序列化, 除此以外我们还需要序列化 row 中 field 的个数, 并且使用 null bitmap 记录是否为 null, 即采用 bool 数组来判断每个 field 是否为 NULL。

反序列化只需逆着序列化的过程即可。

相关函数有:

- `Row::SerializeTo(*buf, *schema)`
- `Row::DeserializeFrom(*buf, *schema)`
- `Row::GetSerializedSize(*schema)`

2. Column 的正反序列化:

Column 记录了一个字段的所有信息: 字段名, 字段类型, 是否唯一等。所以, 序列化时, 将 column 类中所有信息序列化即可。

反序列化只需逆着序列化的过程即可。

相关函数有:

- `Column::SerializeTo(*buf)`
- `Column::DeserializeFrom(*buf, *&column)`
- `Column::GetSerializedSize()`

3. Schema 的正反序列化:

Schema 由一个或多个 Column 组成, 序列化过程总体与 row 较为相似, 不过不需要空位图, 因为 column 一定不为空。

反序列化时, 先读出字节流中 column 的数量, 在一次序列化每个 column 即可。

相关函数有:

- `Schema::SerializeTo(*buf)`

- `Schema::DeserializeFrom(*buf, *&schema)`
- `Schema::GetSerializedSize()`

2. 通过堆表管理记录

功能介绍

堆表是一种将记录以无序堆的形式进行组织的数据结构，不同的数据页（`TablePage`）之间通过双向链表连接。堆表中的记录通过 `RowId` 进行定位。`RowId` 记录了该行记录所在的 `page_id` 和 `slot_num`，其中 `slot_num` 用于定位记录在这个数据页中的下标位置。

堆表中的每个数据页（与课本中的 `Slotted-page Structure` 给出的结构基本一致，见下图，能够支持存储不定长的记录）都由表头（`Table Page Header`）、空闲空间（`Free Space`）和已经插入的数据（`Inserted Tuples`）三部分组成，表头在页中从左往右扩展，记录了 `PrevPageId`、`NextPageId`、`FreeSpacePointer` 以及每条记录在 `TablePage` 中的偏移和长度；插入的记录在页中从右向左扩展，每次插入记录时会将 `FreeSpacePointer` 的位置向左移动。

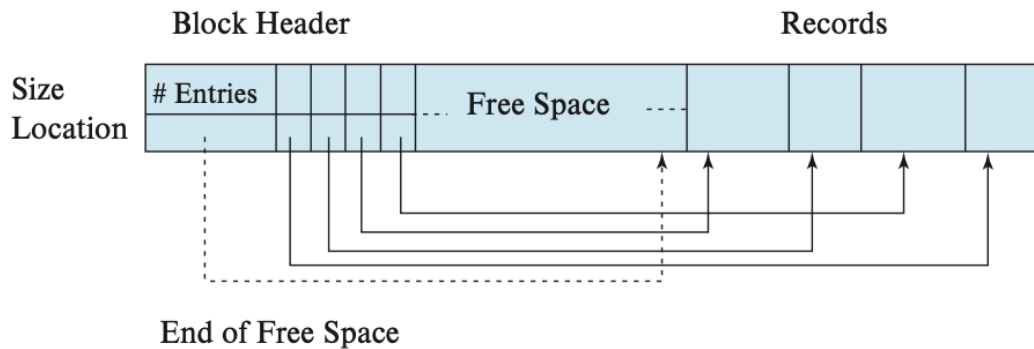


Figure 13.6 Slotted-page structure.

相关函数

1. 堆表的插入

当向堆表中插入一条记录时，一种简单的做法是，沿着 `TablePage` 构成的链表依次查找，直到找到第一个能够容纳该记录的 `TablePage`：即从第一个数据页开始 进行插入，直到插入成功。若均不满足即申请新的数据页

- `TableHeap::InsertTuple(&row, *txn)`：向堆表中插入一条记录，插入记录后生成的 `RowId` 需要通过 `row` 对象返回（即 `row.rid_`）；

2. 堆表的删除

当需要从堆表中删除指定 `RowId` 对应的记录时，框架中提供了一种逻辑删除的方案，即通过打上 `Delete Mask` 来标记记录被删除，在之后某个时间段再从物理意义上真正删除该记录。

- `TableHeap::MarkDelete(& rid, * txn)`：在逻辑上记录它被删除
- `TableHeap::ApplyDelete(&rid, *txn)`：从物理意义上删除这条记录；

3. 堆表的更新

对于更新操作，需要分两种情况进行考虑，一种是 `TablePage` 能够容纳下更新后的数据，另一种则是 `TablePage` 不能够容纳下更新后的数据。

1. 前者在数据页直接更新即可。

2. 后者先将oldtuple删除，再插入newtuple

- `TableHeap::UpdateTuple(&new_row, &rid, *txn)`: 将 RowId 为 `rid` 的记录 `old_row` 替换成新的记录 `new_row`，并将 `new_row` 的 RowId 通过 `new_row.rid_` 返回；

4. 迭代器的实现

堆表中还需要实现迭代器，以便上层模块遍历堆表中的所有记录。迭代器类需要包含 `tableheap` 和 `rowid` 等信息。

另外，迭代器的移动需要注意该row是否为该数据页的最后一个数据，如果不是，则直接移动到下一个数据即可；如果是，则需要翻页到下一页的第一项数据。

而获得迭代器的首尾，需要获得堆表的首迭代器即将堆表的第一页的第一个数据的 `rowid` 传入迭代器初始化即可；尾迭代器只需将 `rowid` 设置为 `invalid` 即可。

- `TableHeap::Begin()`: 获取堆表的首迭代器；
- `TableHeap::End()`: 获取堆表的尾迭代器；
- `TableIterator` 类中的成员操作符
 - `TableIterator::operator++()`: 移动到下一条记录，通过 `++iter` 调用
 - `TableIterator::operator++(int)`: 移动到下一条记录，通过 `iter++` 调用；

2.3 Bonus

本模块的 `bonus` 是优化堆表 (`TableHeap`) 以及数据页 (`TablePage`) 的实现，通过使用额外的空间记录一些元信息来加速 `Row` 的插入、查找和删除操作。

在实现 `bonus` 前的堆表在进行增删查找的时候会从 `first_page_id_` (第一页的id) 开始沿着 `TablePage` 构成的链表依次查找，直到找到第一个能够容纳该记录的 `TablePage` (First Fit 策略)，这样做好处是简单方便，并且不会遗漏每个物理页。但是坏处也非常明显，他就像是一个链表，增删查找都是 $O(n)$ 级的时间复杂度，当数据量大的时候，例如性能测试的10w数据，每次都需要从第一页开始遍历，这非常浪费时间。

一开始的构想是添加一个 `current_page_id_` (当前页的id)，用来指向当前第一个空闲的页，这样每次插入的时候只需要插入在 `current_page_id_` 指向的 `page`，在删除的时候把 `current_page_id_` 指向当前指向的`page`和执行删除操作的`page`中物理位置靠前的那个`page` 即可。但是这样做在删除操作的时候会出现问题：在删除的时候没法判断 `current_page_id_` 到底该指向哪个`page`，因为每个 `page` 的 `id` 大小和物理位置的前后并没有直接的联系。

于是最后选择采用了以下的方案，在堆表中添加一个 `unordered_set<page_id_t> free_page_set_`；用来表示当前空闲页的id的集合，在插入时，每次从集合中取一个id作为`page`的id，在删除时，如果该页本身是未满足的（即已经在集合中），那就直接删除，如果该页本身是满足的（即不在集合中），那就把它添加进集合。

通过该操作，成功让插入操作的时间复杂度从 $O(n)$ 变为 $O(1)$ ，这也是为何我们的插入操作拥有较快的速度，且随着数据量的增长插入的耗时并没有明显的变化（每一万数据的插入时间波动仅为 10^{-2} 秒的级别）。


```
minisql > Query OK, 10000 row affected(1.2890 sec).
minisql >
Query OK, 10000 row affected(1.1980 sec).
minisql > Query OK, 10000 row affected(1.2680 sec).
minisql > Query OK, 10000 row affected(1.3320 sec).
minisql > Query OK, 10000 row affected(1.3510 sec).
minisql > Query OK, 10000 row affected(1.2710 sec).
minisql > Query OK, 10000 row affected(1.3400 sec).
minisql > Query OK, 10000 row affected(1.2750 sec).
minisql > Query OK, 10000 row affected(1.2760 sec).
minisql > Query OK, 10000 row affected(1.3060 sec).
```

具体代码分析见个人报告

框架建议部分见个人报告

3 INDEX MANAGER

3.1 模块概述

Index Manager 负责数据表索引的实现和管理，包括：**索引的创建和删除**，**索引键的等值查找**，**索引键的范围查找**（返回对应的迭代器），以及**插入和删除键值**等操作，并对外提供相应的接口。这一部分需要通过实现一个基于磁盘的B+树动态索引结构，快速随机查找和高效访问数据。

3.2 功能实现

1 B+树数据页

1.1 BPlusTreePage

BPlusTreePage 是 BPlusTreeInternalPage 和 BPlusTreeLeafPage 类的公共父类，它包含了中间结点和叶子结点共同需要的数据，在这个类中，主要实现了对基础数据的**取值与赋值**操作。

- page_type_: 标记数据页是中间结点还是叶子结点；
- key_size_: 当前索引键的长度，
- lsn_: 数据页的日志序列号，目前不会用到，如果之后感兴趣做Crash Recovery相关的内容需要用到；
- size_: 当前结点中存储Key-Value键值对的数量；
- max_size_: 当前结点最多能够容纳Key-Value键值对的数量；
- parent_page_id_: 父结点对应数据页的page_id；
- page_id_: 当前结点对应数据页的page_id。

有许多公共函数，同时一些无法判断是内部节点还是叶子节点的点也可以先用BPlusTreePage 接收。

1.2 BPlusTreeInternalPage

类中间节点 BPlusTreeInternalPage 不存储实际的数据，它只按照顺序存储 **m 个键和 m+1 个指针**，这些指针记录的是子结点的 page_id，而键则记录了 RowId，需要注意的是，RowId的信息包含 pageId 以及 Field 的信息，因此比 pageId大很多。

由于键和指针的数量不相等，因此需要将第一个键设置为INVALID，在进行顺序查找时需要从**第二个键**开始查找。在任何时候，每个中间结点至少是半满的（Half Full）。当删除操作导致某个结点不满足半满的

条件，需要通过合并（**Merge**）相邻两个结点或是从另一个结点中借用（**Move**）一个元素到该结点中（**Redistribute**）来使该结点满足半满的条件。当插入操作导致某个结点溢出时，需要将这个结点分裂（**Split**）成为两个结点。在这个类中我们将上述操作拆分成小的子操作来实现，其中，除了将键值对转移后需要更新父亲节点外，都不涉及 **Fetch** 和 **Unpin** 操作，在更新指针时，仅仅更新本层指针与下层指针，不实现向上更新，向上更新的操作将会在整体操作中通过递归进行。

- 关于 m 的选择：

```
(PAGE_SIZE - 头) / (sizeof(std::pair<GenericKey *, RowId>) - 1)
```

总空间大小除以每一个节点的大小，由于需要先插入再判断是否需要分裂，因此需要减去 1，防止溢出

之前不小心把 RowId 写成了 page_id_t 导致节点过多时溢出，如果发生溢出时，buffer_manager 不会再把新的值储存进这一页，因此会产生意外的错误

1.3 BPlusTreeLeafPage

叶结点 **BPlusTreeLeafPage** 存储实际的数据，它按照顺序存储 **M 个键和 M 个值**，其中键由一个或多个 **Field 序列化** 得到（Field 通过 Column 的来，因此 key 能够表示元组的每一列，key 的大小需要根据列数来进行变化），在 **BPlusTreeLeafPage** 类中用模板参数 **RowId** 表示。叶结点和中间结点一样遵循着键值对数量的约束，同样也需要完成对应的**合并、借用和分裂操作**，操作与内部节点相似，但无需更新下层的父节点，并涉及链表更新，这一部分的操作也将统一在 b+ 树整体操作中实现。

下图为三个类的主要属性和方法：



3.2 B+树索引

索引的创建、删除

1. B+树的创建与删除

1) 初始化

使用初始化函数为b+树赋初值，并将根节点当前 id 设置为无效 id。

相关函数：BPlusTree(index_id_t index_id, BufferPoolManager *buffer_pool_manager, const KeyManager &KM, int leaf_max_size, int internal_max_size)

2) 新建根节点

从内存池中申请一块空间作为叶节点存放根节点的键值对，初始化根节点的 page_id 值，将更新后的索引 id 与根节点 id 插入根索引页，并调用叶节点初始化的函数对当前节点进行初始化，调用 b+树叶节点的insert函数插入根节点。最后 Unpin 该节点，并将它更新到内存。

相关函数: void StartNewTree(GenericKey *key, const RowId &value); UpdateRootPageId(int insert_record) IndexRootsPage::Insert(const index_id_t index_id, const page_id_t root_id)

3) 删除某页

在判断该页确实可以删除后, 调用DeletePage函数进行删除, 如果当前页是叶子页, 则直接删除本页, 并更新链表, 如果有父节点还需要维护父亲(但后续操作中不涉及这一步), 如果没有传入参数, 则直接删除整棵树, 如果是内部节点, 删除该节点并依次删除所有的叶子节点。

相关函数: Destroy(page_id_t current_page_id)

2. 序列化

能够将一个有 k 列的表, m 列为索引键, 将这 m 列的值转换为可传输的字节序列。

Key: 索引键是索引列的值序列化后得到的字符串。索引列的长度作为参数在构造BPlusTreeIndex时作为参数传入, 保存在各个节点中, 方便根据key_size确定每个键值对在模板中的位置, 从而读写。

Value: 值类型可能不同, 叶结点存储RowId

KeyManager: 负责对GenericKey进行序列化/反序列化和比较

插入和删除键值

1. B+树的插入

1) 如果 b+树为空, 则需要初始化根节点, 注意: 为了能够支持 minisql 操作, 为不同索引建立多棵 b+ 树, 需要在每一次更新根节点时将根节点页码与 index_id 一一对应, 这样能够通过 index_id 找到之前建立的 b+ 树进行增删改查操作。

2) b+树非空时

① 通过FindLeafPage 函数 (leftMost = false) 来查找插入的叶节点 id, 如果根节点是叶子节点, 则返回该节点, 如果不是则循环用二分法查找内部节点可以插入的子节点下标, 直到节点为叶子节点。

② 在该叶节点内插入键值。如果键值已经存在, 则返回 false, 如果不存在, 则利用叶节点的 insert 函数将该值插入叶节点的对应位置。

3) 维护节点

当目前叶子节点中键值对的数目大于该节点键值对的最大数目时, 需要进行叶节点分裂。

① 初始化一个新的叶节点, 将满溢的叶节点一半的节点MoveHalfTo新的节点, 并更新叶节点的链表

② 将新的节点插入父亲节点。

如果传入的是根节点, 那么需要建立一个新的根节点, 并将原本的根节点和新的节点指向该节点;

如果传入的是普通内部节点，那么需要先找到它们的父节点，调用InsertNodeAfter 函数将新节点插入父节点的末尾

③ 对更新后的父亲节点进行维护，则递归执行上述操作

相关主要函数：

```
Insert(GenericKey *key, const RowId &value, Transaction *transaction) FindLeafPage(const  
GenericKey *key, page_id_t page_id, bool leftMost) Lookup(const GenericKey *key, const  
KeyManager &KM)  
Split(N *node, Transaction *transaction) InsertIntoParent(BPlusTreePage *old_node, GenericKey  
*key, BPlusTreePage *new_node, Transaction *transaction)
```

2. B+树的删除

1) 等值查找待删除的键值是否存在

2) 通过RemoveAndDeleteRecord 在当前页中删除键值对，如果删除节点之后的 B+树叶节点数目不到最小值，则需要维护

3) 维护节点

① 如果该节点是 root，则使用AdjustRoot来调整节点，如果该节点是叶节点，如果为空直接删除；如果该节点是内部节点则需要根据当前孩子的数量以及合并之后的键值对总数判断是否能够进行合并

② 如果该节点不是 root，则需要判断该节点是否需要进行维护，如果需要合并，则和左兄弟合并（除下标 0），先判断和兄弟合并后是否需要重新分配，需要则可以直接借节点，不需要则进行合并

③ 如果需要重分配Redistribute，则调用 MoveFirstToEndOf 或者 MoveLastToFrontOf 借用节点，并更新父节点的键值对

④ 如果直接合并Coalesce，则调用MoveAllTo来将右边的节点移动到左边，并从父亲节点中删除该节点对应的键值对，此时由于从父亲节点中删除值，因此需要维护父亲节点，递归调用 CoalesceOrRedistribute 重复上述操作。如果是叶子节点还需要维护链表。

主要函数：

```
Remove(const GenericKey *key, Transaction *transaction)  
CoalesceOrRedistribute(N *&node, Transaction *transaction) AdjustRoot(BPlusTreePage  
*old_root_node) Redistribute(LeafPage *neighbor_node, LeafPage *node, int index) Coalesce(N  
*&neighbor_node, N *&node, InternalPage *&parent, int index, Transaction *transaction)
```

索引键的查找

1. 等值查找

如果B+树为空直接返回，如果非空则通过FindLeafPage寻找对应的叶子，再借助Lookup来查找叶子中是否有该键，通过指针传递RowId。

2. 范围查找

通过迭代器实现，如果需要查找所有值，则通过将leftMost 置为 1，能够调用FindLeafPage找到最小值所在的叶子，先找到第一个叶子，顺着链表查找，能够找到最大值所在的叶子。如果需要指定范围查找，则能够通过等值查找的方式找到起始与结尾的叶子，接着便利中间的叶子，将对应的页 id 压入value数组中。

主要函数：GetValue(const GenericKey *key, std::vector &result, Transaction *transaction)，迭代器

3.3 B+树索引迭代器

将所有的叶结点组织成为一个单向链表，然后沿着特定方向有序遍历叶结点数据页中的每个键值对，在BPlusTree类中实现Begin()和End()函数以获取B+树索引的首迭代器和尾迭代器，并适当修改iterator部分以执行获取键值对或递增等操作。

主要函数：Begin(const GenericKey key) End() operator() operator++()

3.4接口

1 BPlusTreeIndex

通过调用BPlusTree 实现

1. 插入

在InsertEntry中，利用 b+ 树的插入接口来向索引中插入新的数据，并维护 key_manager，需要建立一颗新的 b+ 树，此时应该正确储存根节点与对应的index

InsertEntry(const Row &key, RowId row_id, Transaction *txn)

2. 删除

删除某一键值对应的索引，调用Remove 删除 b+树页中的数据，Destroy()直接删除整棵 b+ 树，注意需要同时删除根表中的数据。

RemoveEntry(const Row &key, RowId row_id, Transaction *txn)
Destroy()

3. 查找

判断是单值查找还是多值查找，如果是单值查找则直接调用 b+ 树中的查找功能GetValue 来获取页信息，如果是多值查找，则需要获取其实 key 和终止 key，分别获取 begin 和 end，返回迭代器

ScanKey(const Row &key, vector &result, Transaction *txn, string compare_operator)

2 BPlusTree

相关接口已经在功能实现模块进行具体阐述。

header	header
建树	StartNewTree UpdateRootPageId
删除页	Destroy
插入	Insert
维护插入	Split InsertIntoParent
删除维护	CoalesceOrRedistribute,Coalesce Redistribute
删除	Remove
查找	GetValue
迭代器	Begin End

3.5 整体交互

第三部分与整体模块的交互主要体现在创建新的索引 CreateIndex 以及使用索引进行增删改查上，其中和第四模块 catalog manager 以及第一模块 buffer manager 存在直接交互。

新建一个索引

首先新建一个engine，分析创建索引的语法树（见下图），如果是第一次建立表需要获取columns 并映射到 schema 上（获取初始化 metadata，获取需要建索引的 metaTable 和 metaIndex 的信息），使用语法树中的表名，初始化表上的一个新的索引，如果是打开表则直接使用获取表的函数，总之最主要是要获取包含表信息的 table_info，查询该表上的索引，当符合创建条件时，选择要建立索引的 keys 注意需要至少有一个是 unique 的，接着根据不同的索引类型，调用 InsertEntry 来生成索引。在 catalog 模块储存对应的表名索引号索引名称，在 b+ 树模块储存索引对应的根节点页码。如果在生成索引前已经存在一些元素，则需要为这些元素加上索引。

处理之前的索引

- 1. 使用索引插入
- 2. 使用索引更新

删除索引

首先需要通过根表判断该索引是否通过在删除索引时，需要通过b+树的结构依次递归删除所有该索引申请的页，如果只是删除一个元组，依然需要检查该元组所在表是否存在索引，如果存在，需要删除所有索引中的该元组，防止下一次根据索引查询时出错。

3.6 Bonus

1 利用哈希表建立索引

数据结构

HashTable 用于管理哈希桶，它由 index 值，哈希桶的数量 `tableSize`，缓冲区管理，键值对管理构成。基本思想是一个将 key 通过散列函数进行计算，将哈希值相同的 key 放入同一个哈希桶中，为每一个哈希桶分配一页缓存区，并调用 `buffer_manager` 和 `HashTable` 进行管理。为了能够将 Fetch 到的 Page 成功地转换为哈希桶，定义了一个 Page 类 `HashPage` 储存键值信息，哈希桶的序号，以及当前哈希桶储存的键值数目，防止哈希桶溢出。

采用除留余数法计算哈希桶的序号，并使用分离链接+线性探测法处理数据冲突（这一部分没有使用链表，而是仿照 b+ 树进行数据存取）。`HashIndex` 继承了 `Index` 类，并对虚函数进行重写，调用 `HashTable` 中的函数实现索引的增删改查，但由于哈希表具有一定的局限性，范围搜索并不适用，而单值搜索将会比二叉树索引快速。

每一个索引对应一个哈希表，申请一页用于储存哈希表，并把该页设置成这个索引之后申请的每一页的父亲页，哈希表中记录所有申请页页数，以及以它们的哈希值为下标的页码值，在 `IndexRootsPage` 页中储存索引和对应的哈希表所在页码。

具体代码分析见个人报告
框架建议部分见个人报告

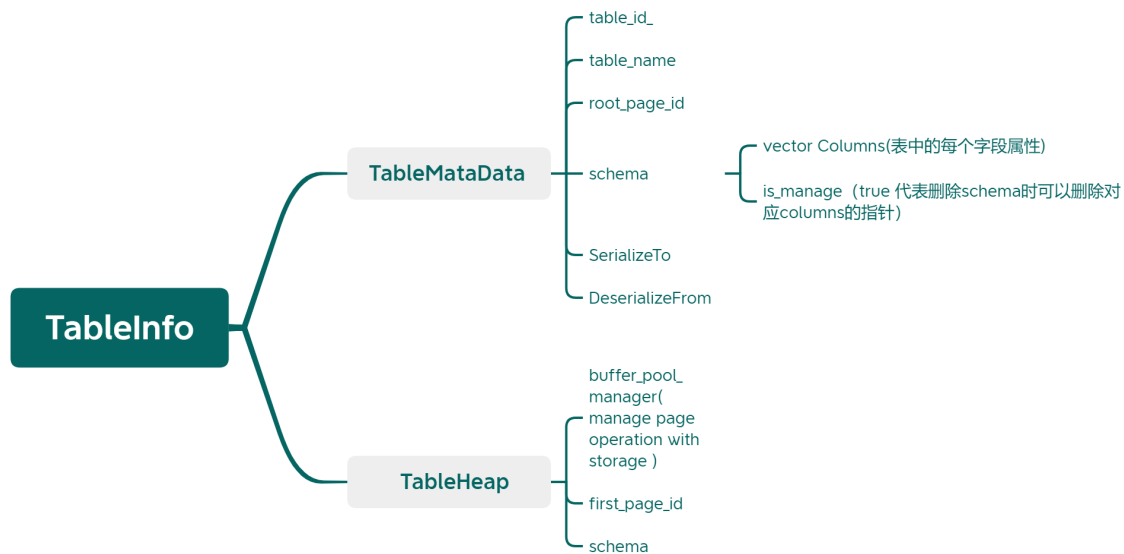
4 CATALOG MANAGER

4.1 模块概述

第四模块用于持久化数据库中表和索引等数据，存储表结构中的字段信息，并提供服务第五部分的函数接口。

4.2 基本功能

4.2.1 TableInfo TableMetaData TableHeap



4.2.2 序列化和反序列化

为持久化存储对象，我们将对象转化成字节序列，可以写进硬盘文件中实现持久化。当需要这些对象的时候，我们可以读取char*类型的字节序列进行反序列化出对象。

CatalogMeta

需要读取该目录中表数量和索引数量，记录转换所需的魔数，并序列化所有的表信息和索引信息

IndexMetadata

新建一个char*类型的buf标记buffer开始记录的位置，通过GetSerializedSize();获取已序列化的字段长度，将索引的id，名字和表id序列化，将表的每列字段信息映射到的索引序列化。最后只要本次增加的序列长度和GetSerializedSize()获取的长度不一致则操作成功。

Row

row由field组成，我们需要存储field的个数并序列化相应的每个field。另外需要用空位图记录每个field是否为空。反序列化的时候按照空位图序列化每个field。

Schema

由于schema的每列是对应相应的字段，所以一定不为空，我们序列化每个存储字段信息的column

4.2.3 目录信息获取和释放

LoadTable LoadIndex

从buffer中获取相应的页面，反序列化里面的信息存储到新建的table或index容器中，调用init函数初始化table_info和index_info，另外，将信息push到CatalogManager中的 tables_ 和 indexes_ 中，所有读取操作完成后，用UnpinPage 释放页面

FlushCatalogMetaPage

在析构CatalogMetaPage的时候会调用。用于将信息序列化后存储磁盘上。首先获取相应的页，序列化信息后释放。另外需要将 `this->tables` 对应的所有表也存储起来。最后调用第一部分的 `FlushPage` 来将页面写进硬盘

DropTable DropIndex

首先确认要删除的index和table是否存在，调用 `Destroy()` 函数删除index，调用 `erase()` 函数删除meta中的相应id

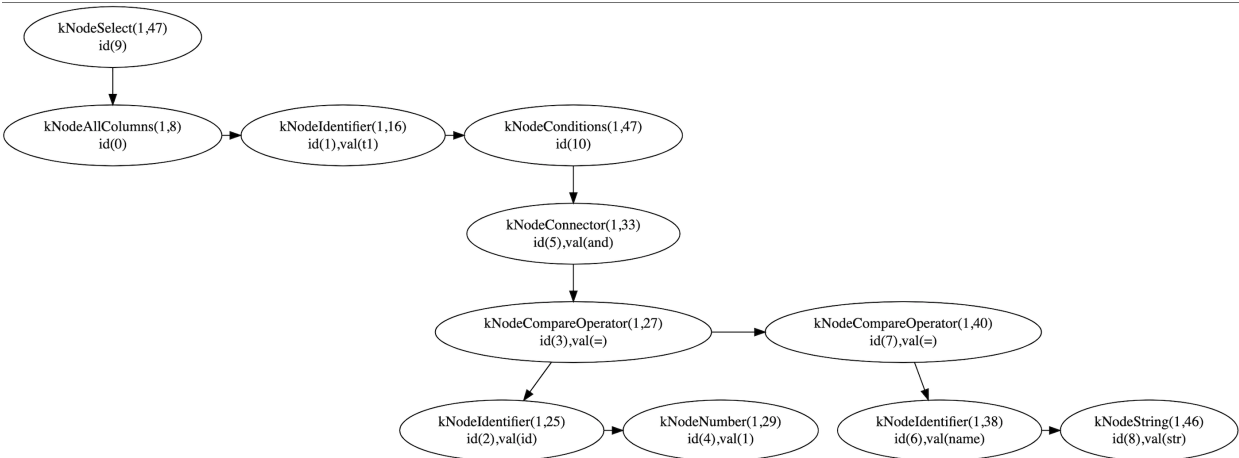
5 PLANNER AND EXECUTOR

5.1 模块概述

本实验主要包括Planner和Executor两部分。Planner的主要功能是将解释器（Parser）生成的语法树，改写成数据库可以理解的数据结构。在这个过程中，我们会将所有sql语句中的标识符（Identifier）解析成没有歧义的实体，即各种C++的类，并通过Catalog Manager 提供的信息生成执行计划。Executor遍历查询计划树，将树上的 PlanNode 替换成对应的 Executor，随后调用 Record Manager、Index Manager 和 Catalog Manager 提供的相应接口进行执行，并将执行结果返回给上层模块。

src/include/parser/minisql.i	SQL的词法分析规则
src/include/parser/minisql.y	SQL的文法分析规则
src/include/parser/minisql_lex.h	<code>flex(lex)</code> 根据词法规则自动生成的代码
src/include/parser/minisql_yacc.h	<code>bison(yacc)</code> 根据文法规则自动生成的代码
src/include/parser/parser.h	Parser模块相关的函数定义
src/include/parser/syntax_tree.h	语法树相关定义

`select * from t1 where id = 1 and name = "str";` 这一条SQL语句生成的语法树如下。以根结点为例说明， `kNodeSelect` 为结点的类型， `(1,47)` 表示该结点在规约（*reduce*，编译原理中的术语）后位于行的第1行第47列（语句末）， `id(9)` 表示该结点的 `id_` 为 9。



在Parser模块调用 `yyparse()` (一个示例在 `src/main.cpp` 中) 完成SQL语句解析后, 将会得到语法树的根结点 `pSyntaxNode`。将语法树根结点传入 `ExecuteEngine` (定义于 `src/include/executor/execute_engine.h`) 后, `ExecuteEngine` 将会根据语法树根结点的类型, 决定是否需要传入 `Planner` 生成执行计划。

5.2 跳转实现

在 `minisql.1` 中, 我们提取输入的sql语句

```
"create" {
    MinisqlParserMovePos(yylineno, yytext);
    return CREATE;
}

"drop" {
    MinisqlParserMovePos(yylineno, yytext);
    return DROP;
}
```

在 `execute_engine.cpp` 中, 会调用 `CreateExecutor` 通过传入的 `plan` 命令构造相应执行器

```
switch (plan->GetType()) {
    // Create a new sequential scan executor
    case PlanType::SeqScan: {
        return std::make_unique<SeqScanExecutor>(exec_ctx, dynamic_cast<const
SeqScanPlanNode*>(plan.get()));
    }
    // Create a new index scan executor
    case PlanType::IndexScan: {
        return std::make_unique<IndexScanExecutor>(exec_ctx, dynamic_cast<const
IndexScanPlanNode*>(plan.get()));
    }
    ...
}
```

然后调用 `execute(ast)` 跳转到相应的函数接口

```
switch (ast->type_) {
    case kNodeCreateDB:
        return ExecuteCreateDatabase(ast, context.get());
    case kNodeDropDB:
        return ExecutedDropDatabase(ast, context.get());
    case kNodeShowDB:
        return ExecutesShowDatabases(ast, context.get());
}
```

在 `main.cpp` 中, 我们首先通过 `inputCommand` 函数读取每行输入, 通过语法分析模块提取关键字, 生成语法树, 最终依靠 `execute` 函数在 `parse` 语法分析结束后进入 `sql` 执行模块

```
auto result = engine.Execute(MinysqlGetParserRootNode());
```

并判断是否达到退出条件

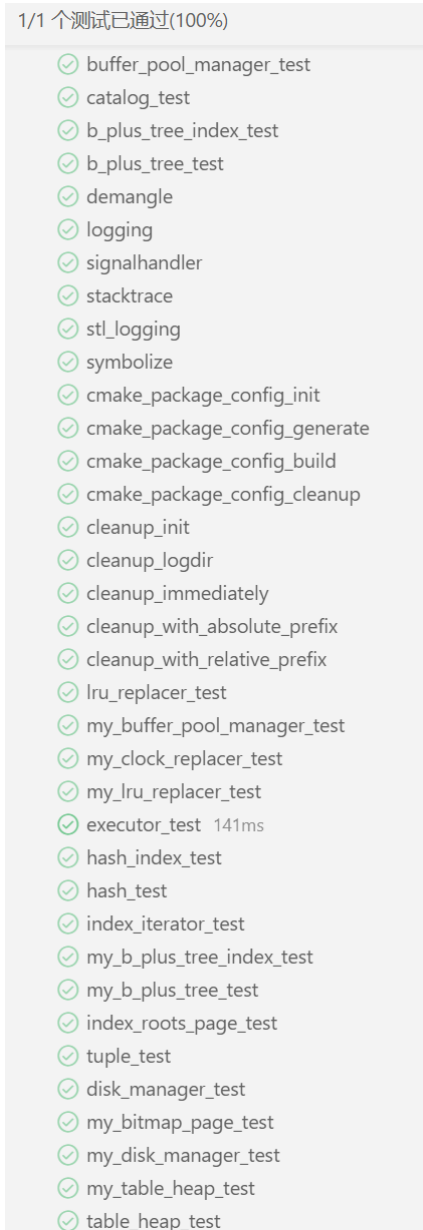
```
engine.ExecuteInformation(result);  
    if (result == DB_QUIT) {  
        break;  
    }
```

具体代码分析见个人报告

§ Chapter 3 – 测试

3.0 正确性测试

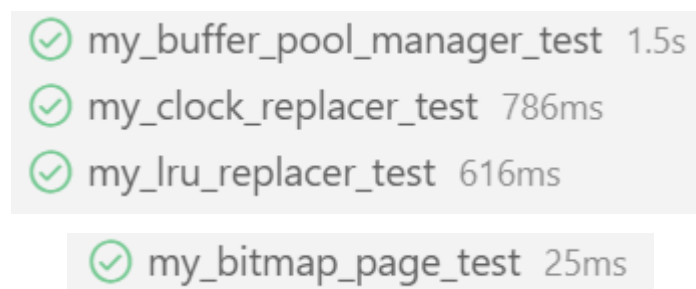
能够通过**所有**的测试点，并且能够通过更大数据量的压力测试与更加完善的综合性测试，在保证正确性的同时也具有较高的性能，在各个部分都进行了优化，如下：



3.1 DISK AND BUFFER POOL MANAGER

本部分添加了 4 个测试文件，分别为 my_buffer_pool_manager_test.cpp , my_lru_replacer_test.cpp , my_clock_replacer_test.cpp , my_bitmap_page_test.cpp , 共计 5 个测试点，涵盖了本模块中的所有函数。

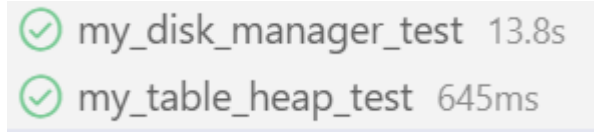
其中， my_buffer_pool_manager_test.cpp , my_buffer_pool_manager_test.cpp , my_bitmap_page_test.cpp , 以及 my_clock_replacer_test.cpp 中的第二个测试点均为压力测试，经测试发现本模块的各个函数在十万级数据的压力下均能正确运行。



3.2 RECORD MANAGER

本部分添加了 2 个测试文件，分别为 `my_disk_managerr_test.cpp` , `my_table_heap_test.cpp` , 共计 4 个测试点，涵盖了本模块中的所有函数。

其中，`my_disk_managerr_test.cpp` 与 `my_table_heap_test.cpp` 中的第一个测试点为压力测试，经测试发现本模块的各个函数在十万级数据的压力下均能正确运行；`my_table_heap_test.cpp` 中的第二第三个测试点为综合性测试，提供了一个模拟真实情况的场景，综合测试了所有的 `table_heap` 相关函数。



```
✓ my_disk_manager_test 13.8s
✓ my_table_heap_test 645ms
```